

¿Cómo funciona nuestro motor de alertas?

Módulo 2

Qué problema resuelve

Este módulo ayuda a decidir, con **datos reales del mismo Excel** que el diagnóstico (30 días, 24 horas, 14 zonas), **cuándo vale la pena avisar** a operaciones y **cuánto sugerir pagar** a los repartidores cuando llueve o cuando el sistema va muy cargado. No son reglas inventadas: se **calibran** a partir del historial y se guardan en un archivo de configuración (`calibration.json`) que el programa lee al vuelo.

Por qué confiamos en estas reglas (la evidencia)

- **Lluvia ligera ya importa.** En el panel, cuando la intensidad supera **0,5 mm/h** en una hora, la probabilidad de ver saturación es del orden de **ocho veces** mayor que cuando la lluvia es más suave o nula. Eso justifica no esperar solo a tormentas grandes.
- **La ciudad no es una sola pieza.** Algunas zonas se complican con poca agua y otras aguantan más. Por eso el motor lleva **números distintos por zona** (coeficientes y umbrales), estimados con la misma lógica que el notebook de Módulo 1.
- **Continuidad M1 a M2.** Puedes volver a generar la configuración desde el Excel con `export_calibration_from_m1`, de modo que alertas y análisis hablen el mismo idioma.

Ideas clave (sin perder rigor)

- **Ratio y saturación.** El **ratio** es pedidos divididos entre repartidores conectados. En el caso, se trabaja con un tramo “saludable” alrededor de **1,2** y se considera **saturación** cuando el ratio supera **1,8**: demasiados pedidos para la capacidad disponible.
- **Sensibilidad por zona (`sensitivity_index`).** Es un puntaje relativo: indica qué zonas reaccionan con más fuerza al subir la lluvia, comparadas entre sí (no es un porcentaje de error del clima).
- **Umbral de alerta (`alert_precip_mm_hr`).** Ver sección siguiente: cómo se justifica con datos y por qué puede variar tanto entre zonas.
- **Pagos sugeridos.** El **pago base** resume lo “normal” (mediana histórica por zona). El **pago recomendado** resume lo que en el pasado se pagaba de media cuando ya llovía por encima del umbral de alerta. El motor combina ambos con el ratio proyectado para proponer **cifras en pesos** (por ejemplo, subir de 55 a 78 MXN), sin pasarse de un **tope del 35 %** sobre el pago base.

Justificación del umbral de lluvia (`alert_precip_mm_hr`)

El umbral no es un número fijo “por criterio de negocio a mano”: se **calcula desde el mismo Excel y la misma clasificación** que el Módulo 1 (columna `clasificacion`, tramo saludable / saturación).

- **Qué pregunta responde.** “¿A partir de qué intensidad de lluvia (mm/h) en el pronóstico conviene tratar el caso como *alertable* para esta zona?”, de forma **coherente con el histórico** donde esa zona ya apareció en saturación operativa.

- **Cómo se estima (por zona).** Se toman las filas del panel donde esa zona está en **saturación**. Sobre esas filas se calcula el **percentil 75** de la precipitación horaria (**PRECIPITATION_MM**). Ese valor resume “cuánta lluvia había en la mayoría de las horas malas”, sin quedarse solo en el caso más extremo (sería inestable) ni en la mediana (demasiado suave si hay colas largas).
- **Por qué a veces sale un número muy bajo o muy alto.** Zonas donde la saturación aparece con poca lluvia tienen umbral bajo (el sistema se estresa con llovizna). Zonas donde hace falta mucha agua para saturar heredan umbral alto: el gatillo solo salta con lluvia fuerte.
- **Respaldo 6,55 mm/h.** Si en esa zona el percentil 75 en horas saturadas es menor que **0,5 mm/h**, el panel sugiere mucha “saturación en seco” (ruido u otros factores) y el percentil ya no discrimina bien. Entonces se aplica un **valor de referencia** del caso empaquetado (misma regla que en `export_calibration_from_m1.py`) para no disparar alertas con casi cero lluvia por artefacto del histórico.
- **Recalibración.** Si cambia el Excel o las reglas de clasificación en M1, se vuelve a generar `calibration.json` y los umbrales se actualizan en bloque; el motor solo lee el JSON.

Qué hace el motor, paso a paso

1. **Mira el clima:** consulta pronóstico horario (Open-Meteo, sin API key) para cada zona. Si el mapa en Excel está incompleto, usa el **punto central** oficial de la zona.
2. **Decide si vale la pena alertar:** toma la lluvia máxima esperada en una ventana corta (por defecto unas 2 horas). Si está **por debajo** del umbral de *esa* zona, no molesta al equipo: no hay alerta.
3. **Proyecta la carga:** combina la lluvia con el coeficiente histórico de la zona (parte fija 1,05 más el efecto de la lluvia, con piso razonable).
4. **Etiqueta el riesgo** en cuatro niveles (**BAJO, MEDIO, ALTO, CRÍTICO**) mezclando ese ratio proyectado con qué tan fuerte llueve comparado con el gatillo de la zona. Los cortes exactos están en `classify_risk_expert` dentro de `decision_engine.py`.
5. **Sugiere MXN** y puede señalar otras zonas a vigilar.
6. **No satura de mensajes:** aplica reglas de *debounce* (no reenviar el mismo evento si nada empeora de verdad), tiempo de espera (**TTL**) y, si se configura, un tiempo mínimo entre alertas de cualquier zona.

Dos zonas opuestas (ejemplo del caso)

Idea	Santiago (más frágil)	Mitras Centro (más resistente)
Sensibilidad a la lluvia	Alta (índice 1,0 en el JSON de referencia): con poca agua ya se mueve la operación.	Baja (alrededor de 0,26): el patrón local aguanta mejor el agua.
Gatillo de alerta	0,55 mm/h: se activa con llovizna.	6,55 mm/h: hace falta lluvia fuerte para pasar el filtro.
Orden de magnitud (mm/h)	Unos 3,1 mm/h para acercarse al “rojo”.	Unos 11,9 mm/h: necesita mucha más agua para un salto parecido.
Qué suele pedir el mensaje	Subir incentivos antes.	Mantener pagos base salvo tormenta seria.

Cómo leer los cuatro niveles de riesgo

- **BAJO:** hay presión, pero todavía lejos de lo peor según las reglas del motor.
- **MEDIO:** conviene monitorear y preparar acciones.
- **ALTO:** priorizar respuesta operativa.

- **CRÍTICO:** combinación muy mala (lluvia extrema o ratio proyectado muy por encima de saturación); el incentivo puede acercarse al **tope del 35 %** respecto al pago base.

Anti-fatiga (por qué no llenan de avisos al coordinador)

Si la severidad no sube y la lluvia no empeora de forma clara, **no se reenvía** el mismo aviso dentro de un tiempo (debounce). Tras un aviso, la zona espera un periodo (**TTL**) antes de repetir avisos parecidos. Opcionalmente se puede exigir un tiempo mínimo entre alertas de *cualquier* zona.

Si cambian los datos

Open-Meteo entrega lluvia horaria coherente con el Excel. Si se actualiza el panel, se puede ejecutar `run_alert_engine.py -recalibrate` para regenerar `calibration.json` antes de consultar el clima en vivo.

Glosario

- **MCO:** método usado en M1 para estimar el efecto de la lluvia sobre el ratio dejando fijos hora y calendario.
- **JSON:** formato de texto donde se guardan umbrales y pagos por zona.
- **WKT:** descripción textual de polígonos en el Excel; si viene truncado, se usa el centroide.
- **TTL:** tiempo de espera entre avisos parecidos; va de la mano con el debounce.